

TracePilot 项目汇报 PPT 大纲



第 1 页 — 封面

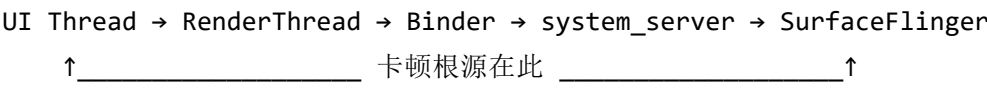
- 项目名称：TracePilot — Frame-Centric 的 Android 调度辅助系统
- 英文副标题：Frame-aligned, dependency-aware scheduling assistant for Android interaction workloads
- 团队：TracePilot（潘智勇、李松茂、邵晨轩、贺小轩、杨子皓）
- 日期：2026 年 6 月
- 一句话定位：在 Pixel 6a 上，通过 eBPF 采集内核调度事件，Perfetto 标定帧边界，构建帧级依赖路径图，识别卡顿根因并输出调度建议



第 2 页 — 项目背景与研究问题

核心问题

Android 交互场景中的卡顿（Jank）严重影响用户体验，根源往往涉及**跨进程等待链**：



传统单进程分析方法无法定位这种跨进程依赖问题。

为什么 PID-Centric 不可行

问题	说明
PID 不稳定	同一 App 每次启动 PID 不同，还会被系统复用
卡顿不是单个 PID 的问题	Jank 根因是跨进程等待链，需全局视角
eBPF 只能观测内核事件	无法直接回答"用户在经历什么"

PID 视角 vs Frame 视角（大字结论）

- ✖

PID 视角：看到 pid=1234 进程 CPU 占用高 → 无法定位根因
- ✔

Frame 视角：看到 f32 帧内 SurfaceFlinger 被 Binder 调用阻塞 12ms → 精准定位

正确路径：Frame-Centric + Dependency-Centric

FrameTimeline 定义问题 → eBPF 提供原因 → Graph 找关键路径 → Hint Engine 做受控干预



第 3 页 — 项目历程与开发路线图

分阶段实施

阶段	内容	状态
前期调研	4 份调研报告/可行性分析，明确技术路线	✔ 已完成
Step 1	FrameTimeline + sched + 身份层 + delay 聚合 + Hint Engine	✔ 已完成
Step 2	Binder/Futex 图 + CPU 频率 + Jank 分类 + 启发式对比 + 视频场景扩展	✔ 已完成
Step 3	Thermal 深化 + Inference-aware + Multi-session 对比	✔ 已完成
Step 3+	Learned policy、Cuttlefish、sched_ext	⏸ 未做 (研究扩展)

老师指导与迭代

- 刑凯老师改进建议：从"全量采集"转向"场景驱动"，聚焦 1-2 个核心场景，建立数据质检流程，做好特征语义映射
- 项目据此调整方向 → 收敛到页面切换 + 视频浏览两大场景

第 4 页 — 前期调研成果（第 1 个月）

调研报告

报告	内容
调研报告（4/4）	Android 16 eBPF 预测调度方案，LLM/自研时序模型 + 调频指令注入内核
可行性报告	eBPF 数据采集 7 层模型设计（统一上下文层 → 可扩展观测层）
补充调研报告	系统行为观测扩展（网络、传感器）、高频场景扩展、关键线程识别方法论
TracePilot 调研扩展	Page Turning、Feed Scroll 场景的行为特征分析

核心结论

- eBPF 适合承担内核态行为观测，最关键是调度事件 + Binder + Futex + CPU 频率
- 一期聚焦页面切换，不承诺网络栈/文件系统/IRQ 等扩展观测
- 采用 Frame-Centric 而非 PID-Centric 的观测视角

第 5 页 — 技术栈总览

技术	角色	版本/详情
eBPF	内核态数据采集	kprobe / tracepoint 挂载，C 编写 BPF 程序
Perfetto	帧边界标定与 Jank 识别	Ground Truth，SQL 查询帧信息
Docker + NDK r26b	交叉编译 ARM64 eBPF 探针	Ubuntu 22.04 镜像封装
Python + sklearn	特征提取 / 自动标注 / 决策树训练	6 维特征，导出 C 头文件嵌入 loader
C (loader)	用户态加载解析	eBPF 加载 + ringbuf 读取 + 图构建 + 推断引擎

实验设备：Pixel 6a (Android 14, Magisk root)

第 6 页 — eBPF 探针设计

为什么选这些探针？（推导逻辑）

用户感知卡顿 → 帧掉 → 线程没跑 → 三种可能：调度排队 / 等 Binder / 等锁
→ 因此必须同时采集 sched_switch + binder + futex
→ 再加 cpu_frequency + thermal 解释环境因素

探针一览

探针	挂载点	采集内容	用途
sched_switch	tracepoint	线程切换 prev/next TID、运行时长、runnable delay	计算就绪等待延迟
sched_wakeup	tracepoint	唤醒延迟	wakeup-to-run latency
binder_transaction	kprobe ⚠️	Binder IPC 调用发起/接收	跨进程依赖分析
futex wait/wake	tracepoint	锁等待	同步阻塞识别
cpu_frequency	tracepoint	CPU 频率变化	大小核调频分析
thermal_temperature	tracepoint	温控温度	降频归因

⚠️ binder 用 kprobe 而非 tracepoint：Android GKI 内核未暴露 binder tracepoint

跨平台编译：Docker → clang 交叉编译为 BPF 字节码 → bpf() 系统加载 → ringbuf 输出 events.bin

第 7 页 — 覆盖的四大场景

场景	测试应用	数据量	分析特点
页面切换 (基础版)	QQ	最高 690MB events.bin	基础 eBPF + Perfetto 帧分析，IRQ/softirq 辅助

场景	测试应用	数据量	分析特点
页面切换+视频浏览 (增强版)	微信 / 抖音	459~451 MB events.bin	双场景对照，Binder/Futex 图， Jank 分类，Hint Engine
信息流滚动	Chrome	261 万事件 / 34s	秒级聚合 + 34 线程级汇总 + 补充 ftrace 分析
相机场景	Google Camera	—	全自动 Pipeline: 编译→部署→采集 →分析→报告

补充：QQ 行为特征分析

- 采集 behavior_features.csv （578 行），按秒级窗口 + 包名聚合
- 分析 QQ 主包 com.tencent.mobileqq 的突发行行为模式（P90 阈值 = 36 事件/秒）
- 识别系统侧并发干扰（系统服务、后台应用对主场景的影响）

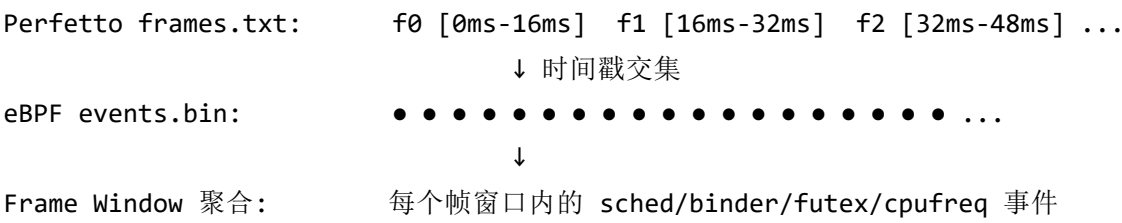


第 8 页 — Frame-Centric 对齐算法

核心问题：eBPF 事件如何与帧对齐？

Perfetto 帧时间线提供帧边界（expected_start / expected_end / actual_end），但 eBPF 事件是独立采集的纳秒级内核事件流。两者必须对齐才能回答"这个 jank 帧内发生了哪些内核事件"。

对齐方法



- 每个 Jank 帧独立开窗，收集该窗口内所有 TID 的调度事件
- 超出窗口的事件（如前一帧残留的 binder 调用）通过依赖继承机制保留
- 计算关键指标：**帧内 runnable delay 总和、Binder 调用深度、Futex 竞争强度**



第 9 页 — 图构建算法：从原始事件到依赖图

节点构建（谁是图中的节点？）

- 图中**每个节点 = 一条线程**（TID）
- 字段： id (TID) / comm (线程名) / role (角色分类) / pid / frame_window_overlap (帧重叠度)
- 角色识别算法（`classify_thread()`，12 类）：

角色	判定依据
UI Thread	TID == target_pid 或 comm 以 "com." 开头
RenderThread	comm 包含 "renderthread" 或以 "rend" 开头
SurfaceFlinger	comm 包含 "surfaceflinger"
Binder RPC / HwBinder RPC	comm 包含 "binder"
GPU Worker	comm 包含 "gpu" 或 "gl"
KernelWorker	comm 以 "kworker" 或 "swapper" 开头
... (共 12 类)	...

边构建（什么是依赖？）

边类型	来源事件	构建逻辑
BINDER_CALL	binder_transaction	from=TID(TX方) → to=TID(RX方)
FUTEX_WAIT	futex	同一 uaddr 上 wait → wake 的线程间关系
SCHED_DEPENDENCY	sched_wakeup	wakee 被 waker 唤醒
DECODE_DEPENDENCY	视频帧解析	解码线程 → 渲染线程的依赖
RESOURCE_STALL	cpu_frequency	CPU 资源不足时的跨线程链

CriticalScore 算法

核心公式：多维度加权评分

$$\text{CriticalScore}(T) = \alpha \times \text{CriticalPosition}(T) + \beta \times \text{RunnableDelayShare}(T) + \gamma \times \text{ConnectionDegree}(T)$$

直觉解释（为什么是这三个维度？）：

维度	含义	直觉
CriticalPosition(T)	线程在关键路径上的深度	越靠近帧渲染末端越关键
RunnableDelayShare(T)	该线程 runnable delay / 总 delay	占帧内浪费时间的比例越大越关键
ConnectionDegree(T)	Binder/Futex 连接数(出度+入度)	影响其他线程越多越关键

默认权值： $\alpha=0.4$, $\beta=0.4$, $\gamma=0.2$ （可调）

图可视化方法

- 分层布局：BFS 计算 depth → 同层按 CriticalScore 降序排列
- 三层子图：graph_binder.svg (仅 Binder 边) / graph_futex.svg (仅 Futex 边) / graph_critical.svg (全图+关键路径高亮)
- 颜色编码：每种角色映射固定颜色（SurfaceFlinger=红, UI=紫, Render=蓝, GPU=橙...）



第 10 页 — 核心分析成果（页面切换 + 视频浏览）

三次采集对照

维度	页面切换 Run 1	页面切换 Run 2	视频浏览
总帧 / Jank	2171 / 1575	1271 / 1228	2141 / 1524
Jank 率	72.5%	96.6%	71.2%
VD 帧	0	0	561
图规模	6968 节点 / 5234 边	4799 节点 / 3106 边	6622 节点 / 4992 边
边类型分布	BINDER=52, FUTEX=2371	BINDER=53, FUTEX=1770	BINDER=407, FUTEX=1436
调频抑制比	0.00	0.47	1.00
最高温度	31.4°C	41.5°C	52.3°C

根因推断

场景	主导根因	推荐 Hint	置信度
页面切换 Run 1	RUNNABLE_DELAY	PROTECT_UI_CHAIN → surfaceflinger	0.9999
页面切换 Run 2	RUNNABLE_DELAY	UCLAMP_MIN_TEMPORARY (降频抑制)	0.9999
视频浏览	RUNNABLE_DELAY	BOOST_THREAD	0.9999

关键发现与洞察

数据	洞察
Run1 Jank 率 72.5%，温控 0.00	→ 即使没有温度问题， 调度竞争本身 就是主要瓶颈
Run2 Jank 率 96.6%，温控 0.47	→ 温度升高后降频介入，Jank 率进一步恶化—— 温控是雪上加霜
视频 Jank 率 71.2%，温控 1.00	→ 温控最严重但 Jank 率并非最高—— 视频场景有额外延迟容忍

- Top-5 嫌疑线程中包含 rcuop、kworker 等系统线程，说明卡顿涉及系统级资源竞争



第 11 页 — Step 1：基础能力实现

已完成模块

#	功能	实现文件
1	Perfetto FrameTimeline	frame_query.sql → SF/VD/VF/AP 帧
2	sched_switch / wakeup 采集	BPF + events.bin v3
3	身份解析	identity.c：Session / ProcessInstanceId / ThreadKey / sidecar JSON
4	Frame window delay 聚合	Phase 5b 用户态重算 runnable delay + wakeup

#	功能	实现文件
5	UI/RenderThread 角色识别	classify_thread() 12 类
6	Top-K 关键线程输出	-G -k N → result.json
7	Hint Engine	hint_engine.c : BOOST / UCLAMP / PROTECT + TTL + audit

Hint Engine 安全机制详解

Hint Engine 的设计遵循"安全优先"原则，确保即使 hint 注入不当也不会导致系统崩溃：

安全机制	实现方式
TTL（Time To Live）	每个 hint 有默认过期时间（如 16ms = 1 帧），超时自动回滚，避免 hint 长期生效
自旋保护	检测到线程自旋（5ms 内连续 3 次 on CPU 无 sleep），自动撤销其 UCLAMP hint
线程黑名单	kworker、rcuop、swapper 等系统内核线程不可被 hint 干预
置信度阈值	hint 只有 inference confidence > 0.7 时才会被提交；建议 (< 0.5) 只记录不执行
audit 日志	每条 hint 的"目标 TID、操作类型、时间戳、预期持续时长、实际持续时长"全部记录在 result.json



第 12 页 — Step 2：增强能力实现

Binder / Futex 图分析

- Binder 图：展示跨进程 IPC 依赖关系，识别 Binder 瓶颈
- Futex 图：展示锁竞争关系，识别 Futex 阻塞
- 图可视化：自动生成 SVG 图（graph_binder.svg、graph_futex.svg、graph_critical.svg）

Jank 分类器

- 6 维特征空间 → 决策树分类（sklearn DecisionTreeClassifier）

- 自动标注（`auto_label.py`）→ 可疑帧筛选（`suspect_frames.py`）→ 人工复核 → 模型训练
- 分类结果嵌入 C 头文件（`learned_model.h`），loader 内直接运行
- 评估方式：LeaveOneOut 交叉验证 + confusion matrix

Camera 场景：延迟分解方法

Camera 场景实现了最精细的延迟归因，将 Top-K 线程的总阻塞时间拆解为三大组成部分：

总阻塞时间 = 调度竞争（Runnable Delay） + Binder IPC 等待 + Futex 锁等待

- **角色识别**：基于 comm 字段的 12 类线程角色自动标注
- **DAG 关键路径构建**（`critical_path.py`）：以帧为窗口，BFS 分层，ThreadKey 去重
- **根因归因**（`root_cause.py`）：每个 Top-K 线程的延迟百分比分布
- **安全调优**（`safe_hint_engine.py`）：黑名单过滤（`system_server/surfaceflinger` 等）+ 置信度阈值 0.6 + adb shell 命令生成

信息流滚动：两种聚合策略对比

聚合策略	适用场景	压缩效果	代表字段
帧级窗口（页面/视频）	需要帧精确对齐	帧数级（~2000行）	runnable_delay, 边分布
秒级窗口（信息流）	快速观察宏观趋势	秒数级（~35行）	total_events, sched_switch 频次

综合报告对两种策略进行了对比分析，指出秒级窗口适合概览，帧级窗口适合微观归因。



第 13 页 — Step 3：深化与对比

Thermal 深化分析

- 硬件层读取系统温度传感器，通过 thermal tracepoint 采集
- 与 cpu_frequency 联动计算 **freq_throttle_ratio**（降频抑制比）
 - 0.0 = 无降频，1.0 = 完全限频
- 视频场景实测 `freq_throttle_ratio` = 1.00（温度 30.3→52.3℃）
- 页面切换场景 Run 2 实测 `freq_throttle_ratio` = 0.47（温度 36.8→41.5℃）

- 温控降频被识别为独立的 Jank 根因类别（ THERMAL_THROTTLE ），在热负载场景中贡献显著延迟

Multi-session 对比

- 通过 `--compare-dir` 参数指定多个输出目录，生成 `compare_report.json`
- 对比维度：Jank 率、根因分布、图规模、Top-5 关键线程、thermal profile、freq_throttle_ratio
- 页面切换 Run 1 vs Run 2 vs 视频浏览的三次采集对照：
 - Jank 率差异（72.5% → 96.6% → 71.2%）与温控降频程度直接相关
 - 页面切换 Run 1 无温控（0.00）但 Jank 率仍高达 72.5%，说明调度竞争本身即显著
 - 视频场景受温控影响最大（1.00），但 Jank 率与页面切换 Run 1 相近，暗示视频场景的延迟容忍度可能更高

Cross-scenario 分类器验证

- 页面切换场景（6 维特征）与视频场景（增加 decode_late 为第 7 维）使用同一套分类流程
- 自动标注实验结果验证了**7 维特征在视频场景中的必要性**：若无 decode_late 特征，系统将误将视频解码延迟归因为 RUNNABLE_DELAY
- 分类器通过**困惑度（Perplexity）**评估输出质量，确保模型泛化能力

Inference Engine 证据链

信号	来源	权重范围	用途
runnable_delay	sched_switch	0.0~1.0	调度竞争主信号
binder_centrality	binder_transaction	0.0~1.0	跨进程 IPC 瓶颈
futex_wait	futex	0.0~1.0	锁竞争强度
thermal_throttle	thermal + cpu_frequency	0.0~1.0	温控降频
decode_late	视频帧解码时间	0.0~1.0	视频解码延迟
system_irq	irq/softirq	0.0~1.0	中断扰动

多信号加权融合 → hypothesis + confidence → hint 映射（BOOST/UCLAMP/PROTECT）



页面切换-视频浏览增强版

脚本	功能
deploy.sh / deploy.ps1	一键部署到 Pixel 6a 并采集数据
frame_query.sql	Perfetto SQL 查询帧信息
thermal_query.sql	温度数据提取
graph_features.py	从图拓扑中提取每帧的边类型分布特征
trace_features.py	从 trace 中提取时序特征
trace_label.py	基于规则的自动标签生成
auto_label.py	启发式自动标注 Jank 根因
label_jank.py	人工标注辅助工具
suspect_frames.py	筛选可疑帧（标注可能错误的帧）
train_jank_model.py	决策树分类器训练 + C 头文件导出
export_step2_graphs.py	图可视化导出（SVG）
render_graph_svg.py	SVG 渲染

相机场景

脚本	功能
auto_run.py	全自动：编译→部署→采集→拉取→分析→报告
analyze_delays.py	延迟聚合 + Binder 配对 + Futex 统计
critical_path.py	DAG 关键路径 + 评分
root_cause.py	根因归因
safe_hint_engine.py	安全调优配置 + shell 脚本
generate_report.py	生成 MD 报告

我们做了什么

#	成果
✓	eBPF + Perfetto 双通道采集：覆盖页面切换、视频、信息流、相机四大场景
✓	帧对齐 + 依赖图构建：~7000 节点 / ~5000 边的关键路径图
✓	6 维特征 + 决策树分类：端到端 Jank 根因自动分类 Pipeline
✓	Hint Engine：安全调度建议（TTL / 自旋保护 / 黑名单 / 置信度阈值）

我们发现了什么

发现	含义
调度竞争是卡顿主因	无温控条件下 Jank 率仍高达 72.5% — 非 Binder 非锁，就是调度问题
温控降频雪上加霜	温度升高后 Jank 率升至 96.6%，降频介入显著恶化
场景差异不容忽视	视频温控最严重但 Jank 率非最高 — 不同场景延迟容忍度不同
Hint Engine 自适应	能根据温控程度自动切换策略（PROTECT → UCLAMP）

核心贡献

证明了 Frame-Centric（帧对齐 + 依赖感知）比 PID-Centric 更有效地定位卡顿根因

下一步

方向	说明
 sched_ext	可编程调度器，更灵活的内核调度注入
 Learned Policy	基于强化学习的调度策略学习
 多场景扩展	游戏 / 支付等高频交互场景



第 16 页 — 附录：关键技术指标

指标	页面切换 Run 1	页面切换 Run 2	视频浏览	信息流滚动
采集时长	—	—	—	34.2s
原始事件数	~870万	~580万	~500万	261万
events.bin 体积	690 MB	459 MB	451 MB	—
图节点	6968	4799	6622	—
图边	5234	3106	4992	—
人工标注帧	—	—	—	1575+帧已标注
Jank 率	72.5%	96.6%	71.2%	—

说明： 以上为完整 PPT 大纲，共 15 页。每页对应一张幻灯片，可根据实际汇报时间长短选择精讲或概讲。推荐重点讲第 2 页（背景）、第 5~8 页（技术与数据 Pipeline）、第 9 页（核心成果）、第 12 页（Step 3 扩展），其余作为补充。